Guía de paso a producción — ComfyUI "Everything-Ready" para Vast.ai

Actualizado: 2026-11-04 · **Fuente de código:** rama `main` (verifica el estado de CI del commit actual antes de desplegar). **Público objetivo:** operadores desde Windows, macOS o Linux; los pasos con helper Bash se señalan explícitamente.

A. Qué tienes ahora

Tienes una imagen Docker Linux lista para producción que convierte una GPU RTX 5090 o RTX 4090 alquilada en Vast.ai en un ComfyUI protegido, accesible desde un navegador en Windows, macOS o Linux. La imagen incluye ComfyUI con 12 custom nodes preinstalados, un catálogo de modelos por fases, el portal `/home`, **Comfy Studio** en `/studio/` (plantillas, proyectos, percepción, transcripción/diarización, agente, galería, recuperación durable de jobs, webhooks y EDL), estado/diagnóstico en `/status` y el supervisor Rust `comfyctl`. El helper `tools/mac/comfy` es opcional y funciona desde una shell compatible en macOS/Linux; en Windows puede usarse dentro de WSL2. No se requiere Mac para usar la interfaz web. Consulta el índice documental y la arquitectura para los límites de plataforma y componentes.

> **Qué es real y qué es simulado en Comfy Studio.** La extracción de keyframes, `sqlite-vec`, el almacenamiento de identidades, el EDL y el render final son componentes reales. El backend ML arranca en `auto`: usa modelos reales si el probe de capacidades tiene éxito; de lo contrario usa simulación determinista, declarada por `simulated: true` y `simulation_notice`. `STUDIO_PERCEPTION_BACKEND=real` exige modelos reales y falla de forma explícita cuando no están disponibles; `sim` fuerza el camino simulado. Algunas funciones del índice visual CLIP permanecen deterministas incluso con backend real. Comprueba siempre el campo de respuesta antes de evaluar calidad. Si falta `sqlite-vec`, las funciones dependientes se degradan y reportan el motivo sin tumbar todo Studio.

B. Checklist de una tarde

1. Crear el repo en GitHub y subir el código — YA HECHO en este repo

> El remote `origin` ya existe y el código está empujado en `main` con CI en verde. Haz este paso **solo si empiezas de cero** con otro repo.

Qué harías (caso desde cero): crear un repositorio nuevo (vacío, sin README) en GitHub y empujar el código:

```
# Sustituye <tu-usuario> por tu usuario de GitHub
git remote add origin https://github.com/<tu-usuario>/comfyuipod.git
git push -u origin main
```

Cómo sé que va bien: el push termina sin errores y al refrescar la página del repo en GitHub ves tus archivos (`Dockerfile`, `comfyctl/`, `scripts/`…).

2. Esperar el primer workflow verde — YA HECHO en este repo

> El workflow ya corre en verde en cada push a `main`.

Qué hace: en la pestaña **Actions** del repo, el workflow *"Build, Validate and Publish ComfyUI Vast.ai Image (GHCR)"* valida los scripts, compila la imagen Docker, le hace un smoke test y la publica en GHCR. **Medido en CI: ≈ 10.5 min** el job de build reutilizando la caché `buildcache`; la primera ejecución sin caché tarda bastante más.

Cómo sé que va bien: el workflow aparece con un ✔ verde. Si falla, abre el job rojo y lee el paso marcado con ✘: cada nombre de paso indica qué se validaba (ShellCheck, Hadolint, SHAs de nodos, disponibilidad de modelos en HuggingFace, tests de Rust…).

3. Hacer público el paquete en GHCR

Qué haces: para que Vast.ai descargue la imagen sin credenciales, ve a tu perfil de GitHub → **Packages** → **comfyuipod** → **Package settings** → **Change visibility** → **Public**. El nombre final de la imagen será:

```
ghcr.io/<tu-usuario>/comfyuipod:latest
```

Confirma el nombre exacto tras el primer push (aparece en la página del paquete; GitHub lo pone en minúsculas).

Cómo sé que va bien: abres en una ventana de incógnito `https://github.com/<tu-usuario>/comfyuipod/pkgs/container/comfyuipod` y la página del paquete se ve sin iniciar sesión.

4. Crear cuenta en Vast.ai y añadir saldo

Qué haces: registrarte en cloud.vast.ai, verificar la cuenta y cargar saldo. Con **$5–10** te sobra para todas las pruebas de esta guía (una sesión de aceptación de una hora cuesta céntimos).

Cómo sé que va bien: en *Billing* ves tu saldo disponible y en *Account* puedes copiar tu **API Key**.

5. Preparar el cliente (Vast.ai + helper opcional `comfy`)

Qué haces: instala y autentica la CLI de Vast.ai si usarás el helper. `tools/mac/comfy` es Bash y se soporta en macOS y Linux; en Windows ejecútalo dentro de WSL2. El navegador para `/home`, ComfyUI y `/studio/` funciona desde los tres sistemas. En Windows nativo puedes operar desde la consola web de Vast.ai y el navegador, o usar WSL2 para los comandos de esta guía; el script Bash no es un ejecutable PowerShell.

```
pipx install vastai          # (fallback: pip3 install --user vastai)
vastai set api-key <TU_VASTAI_API_KEY>

mkdir -p ~/.local/bin
cp tools/mac/comfy ~/.local/bin/comfy
chmod +x ~/.local/bin/comfy
```

Añade a tu archivo de inicio de shell si no está ya (por ejemplo `~/.zshrc` en zsh): `export PATH="$HOME/.local/bin:$PATH"`

Cómo sé que va bien: `comfy all` lista tus instancias (ninguna aún) sin errores de autenticación.

6. Apuntar el helper a tu imagen

```
export COMFY_IMAGE=ghcr.io/<tu-usuario>/comfyuipod:latest
```

Hazlo persistente añadiéndolo también al archivo de inicio de tu shell (p. ej. `~/.zshrc` en zsh):

```
echo 'export COMFY_IMAGE=ghcr.io/<tu-usuario>/comfyuipod:latest' >> ~/.zsh
```

Cómo sé que va bien: `comfy rent --dry-run` muestra la imagen correcta y el resumen de variables de entorno definidas (contraseña, `HF_TOKEN`, reenviadas opcionales) antes de pedir confirmación.

7. Primera sesión de aceptación

Qué haces:

```
comfy rent general   # perfil general: 27.75 GiB de modelos, buena prueba
comfy open           # espera al arranque y abre el portal /home
comfy report         # descarga /report.md y lo copia al portapapeles
```

El helper te muestra el **Top 5 de ofertas RTX 5090 (o RTX 4090 si no hay 5090)** con **el precio por hora real antes de confirmar**: ahí ves exactamente lo que vas a pagar. El dry-run muestra además la imagen y el resumen de variables de entorno definidas (enmascaradas).

Cómo sé que va bien: primero ves la pantalla de espera (Fase 1 descargando los modelos ligeros, 1-3 min según red, tope 10); el portal `/home` carga con sus tarjetas; `/status` muestra GPU, VRAM y progreso de descargas; y al cabo de unos minutos ComfyUI abre y puedes arrastrar un workflow. Pulsa **R** en ComfyUI para refrescar los desplegables cuando termine la Fase 2 (~4–8 min a 1 Gbps / 500 Mbps en perfil `general`).

> La lista de verificaciones completa de esta primera sesión está en `docs/auditorias/AUDITORIA_AGENTE_V5.md` (sección 3). De esa checklist, CI ya cierra en cada push verde: **V1** (build sin errores), **V6** (sin `IMPORT FAILED`), **V7** (`hf` / `hf_xet` correctos) y **V12** (limpieza de `user/`); la publicación del paquete GHCR como **Public** es el paso 3 de esta guía.

8. Verificaciones clave de aceptación

1. **Sesión real con descargas:** en `/status`, la Fase 2 avanza y los checkpoints del perfil `general` aparecen en ComfyUI tras pulsar `R`.
2. **FATAL simulado:** alquila una instancia con una GPU pequeña (menos de 20 GB de VRAM, p. ej. una RTX 3060) — **créala a mano con `vastai create instance` (el helper filtra `gpu_ram ≥ 24` y no puede alquilar GPUs pequeñas)**: el arranque estricto debe mostrar el **banner rojo FATAL** en `/home` y `/status`, y la instancia saldrá con código **20** pasados 10 minutos de gracia. Ejecuta `comfy destroy <id>` en cuanto lo veas para no pagar.
3. **`comfy watch`:** muestra el estado en vivo desde la terminal cada 3 segundos.

4. **Secretos redactados:** `comfy report --stdout` no muestra tu `COMFY_PASS` ni tokens: `comfyctl` los redacta automáticamente en `/report.md`.
5. **Variables con espacios (cierra V27 / S-07) — VERIFICADO:** `comfy rent` cita **solo el valor** (`-e COMFYUI_EXTRA_ARGS="--fast --cpu"`), que es el contrato del parser de vast-cli (limpia comillas del valor, nunca de la clave), así que la clave llega intacta y V27 **pasa**. Ciérralo así: alquila con `COMFYUI_EXTRA_ARGS="--fast --cpu"` exportada y comprueba en `/status/api` que `env_masked` muestra `COMFYUI_EXTRA_ARGS` con su valor correcto.
6. **Runbook S3 en `MODEL_PROFILE=studio` (`comfy rent studio` → `comfy studio`):** en la pestaña **Percepción & EDL** de `/studio/`, sube un clip de vídeo, extrae keyframes con `ffmpeg`, indexa en `sqlite-vec`, registra una identidad, busca planos, transcribe/diariza y renderiza EDL verificado con `ffprobe`. Comprueba `/studio/api/health` y `perception.backend` / `media_tools` para saber qué capacidades están activas. El valor por defecto `STUDIO_PERCEPTION_BACKEND=auto` usa modelos reales solo cuando el probe pasa; si usa fallback, las respuestas lo indican con `simulated: true`. `real` exige backend real; `sim` fuerza simulación. El índice visual CLIP conserva componentes deterministas. No evalúes calidad de modelo a partir de respuestas simuladas. Los pesos `InsightFace buffalo_l` están sujetos a licencia **no comercial [NC]**; revisa las licencias antes de habilitar uso real/comercial.

8.1 Aceptación semántica de shots en producción (PR #32)

La orquestación (`studio/comfy_studio/orchestration/`) ya no se limita a ejecutar: cada shot del plan lleva un criterio de aceptación (`acceptance`, umbrales `*_min`). El evaluador (`evaluator.py`) puntúa el render (negro/parpadeo vs. su `acceptance`); si pasa → **ACCEPTED** y el shot pasa de producción a la galería del proyecto; si no, sube un intento y replanifica con otra plantilla (`template_selector.py`). Consulta el estado por shot en los eventos SSE (`evaluate`) y en `/studio/api` del proyecto. Esto cierra el bucle agente→producción→galería sin revisión manual intermedia.

8.2 Recuperación de jobs y webhooks (opcional)

`comfy rent studio` reenvía estas opciones si las exportas en la terminal **antes de alquilar**. La recuperación durable está activa por defecto: los renders elegibles interrumpidos por un reinicio se reencolan una sola vez, 30 segundos después del arranque de Studio. Para desactivarla, define `STUDIO_RESUME_JOBS=0` ; `STUDIO_RESUME_DELAY_S` ajusta la espera.

Para recibir eventos, configura una URL HTTPS pública y un secreto compartido antes de crear la instancia:

```
export STUDIO_RESUME_JOBS=1
export STUDIO_RESUME_DELAY_S=30
export STUDIO_WEBHOOK_URL='https://tu-receptor.example.com/comfy-events'
printf 'Secreto compartido del webhook: '
read -r -s STUDIO_WEBHOOK_SECRET
echo
export STUDIO_WEBHOOK_SECRET
comfy rent studio --dry-run  # confirma variables; los valores del webhook
comfy rent studio
```

El receptor verifica `X-Studio-Signature` como HMAC-SHA256 del cuerpo JSON usando ese secreto. Usa únicamente un endpoint de confianza: se rechazan URLs que apunten a loopback o redes privadas; no publiques el secreto en `~/.zshrc`, repositorios, capturas ni logs. El helper enmascara URL y secreto en `--dry-run`, y Comfy Studio los redacta en `/status/api` y `/report.md`. Sin webhook configurado, Studio opera normalmente sin enviar notificaciones.

8.3 Envío asíncrono de plantillas

El endpoint de envío de una plantilla responde `202 Accepted` con `job_id` cuando el trabajo queda encolado; no es la confirmación de que el render terminó. Sigue el estado con `GET /studio/api/jobs/{job_id}` o los eventos SSE de `/studio/api/progress`. Los clientes que necesiten el contrato síncrono pueden usar `?wait=1`, aunque para renders largos se recomienda el seguimiento por job. El límite por defecto es `STUDIO_MAX_CONCURRENT_RENDERS=2`; no aumentes `STUDIO_SERVER_WORKERS`, que debe permanecer en `1`.

9. Al terminar: rescatar outputs y destruir la instancia

```
# Primero rescata tus renders a tu máquina (re-ejecutable: omite lo ya des
comfy fetch-outputs <id> ~/Downloads/comfy-renders

# Después destruye la instancia
comfy destroy <id>
```

Cómo sé que va bien: `comfy fetch-outputs` imprime `[OK] fetch-outputs: N archivo(s) descargados…` con la estructura `output/…` en el destino local; `comfy all` ya

no lista la instancia, o la muestra destruyéndose. **El disco sigue facturando aunque pares la instancia**: solo `destroy` detiene todos los cargos.

> **Dos respaldos distintos:** `comfy fetch-outputs` rescata renders de ComfyUI en `output/`; no respalda la base ni los proyectos de Studio. Si quieres conservar `studio.db`, el índice `index.db` y `projects/`, crea y verifica un snapshot portable antes de destruir la instancia siguiendo Copias portátiles de Studio. El archivo no incluye modelos, configuración, secretos ni necesariamente el output de ComfyUI. Para restaurar, detén Studio, conserva una copia del destino actual, verifica el ZIP y solo entonces extrae los datos en `STUDIO_DATA_DIR`.

C. Costes aproximados

Concepto	Coste aproximado	Notas
RTX 5090 (32 GB)	~$0.45–0.95/h on-demand	El precio exacto se ve en `comfy rent` antes de confirmar (varía por oferta)
Disco (120 GB)	Unos céntimos/hora adicionales	El helper alquila con `--disk 120` (suficiente para `all`: 63 GiB modelos + ~16 GB imagen); usa `--disk 150+` a mano si acumulas mucho output. Incluido en la estimación de coste total/hora que muestra `comfy rent`
RTX 4090 (24 GB, fallback)	~$0.30–0.55/h on-demand	El helper la busca automáticamente si no hay 5090
Sesión de prueba de 1 h	Céntimos a $0.95	Según la oferta elegida
Instancia **parada** (no destruida)	El disco **sigue facturando**	Por eso el paso 9 es `destroy`, no “stop”

Regla de oro: **máquina encendida solo mientras la usas; `comfy destroy` al terminar.**

D. Qué hacer si algo falla

Síntoma	Qué significa	Qué hacer
Banner rojo **FATAL** en `/home` / `/status`	El arranque estricto (`strict-check` S1–S10, o S1–S11 en `studio`) detectó un host incompatible (GPU, VRAM, RAM, disco…)	Ejecuta `comfy destroy <id>` y alquila otra oferta; el banner indica la causa y la remediación. La instancia sale sola con código 20 tras 10 min
Pantalla de espera eterna	La Fase 1 está descargando o algo la bloqueó	Abre `/status` y mira el progreso de descargas y la pestaña de logs; el motor de triaje (17 patrones) suele nombrar la causa exacta
Error 401 al abrir el portal	Contraseña incorrecta o no definida	`comfy pass <id>` recupera la contraseña autogenerada desde los logs de arranque
ComfyUI no responde pero el portal sí	El supervisor está reiniciando `main.py` (OOM, crash CUDA, etc.)	Mira `/status` (clasifica la salida: 137 OOM, $^{134}/_{139}$ crash nativo…) y `/report.md` para el informe completo
Faltan modelos en los desplegables	Fase 2 aún en curso	Espera a que termine en `/status` y pulsa **R** en ComfyUI
`comfy rent` falla con la imagen	`COMFY_IMAGE` mal exportada o paquete GHCR privado	Revisa el paso 6 y que el paquete sea público (paso 3)
EDL/keyframes/ASR no encuentra `ffmpeg` o `ffprobe`	El ejecutable no está en el `PATH` que ve Studio	Consulta `/studio/api/health` → `media_tools.ffmpeg_available` / `ffprobe_available` ; instala la herramienta en el entorno de Studio o define `STUDIO_FFMPEG_BIN` /

Síntoma	Qué significa	Qué hacer
		`STUDIO_FFPROBE_BIN` con las rutas completas.
Error de Studio difícil de localizar	Hace falta correlacionar una respuesta con el log	Guarda el `X-Request-Id` de la respuesta. Activa `STUDIO_JSON_LOGS=1` en la configuración de la instancia y busca ese valor en `request_id` de las líneas JSON. Mantén `STUDIO_SERVER_WORKERS=1` ; el servidor rechaza varios workers.
Algo raro sin nombre claro	—	`comfy report <id>`: el informe está pensado para pegárselo a un agente IA y que lo diagnostique

---

E. Notas de seguridad básicas

- **HTTP (no HTTPS):** el portal y ComfyUI van por HTTP en el puerto de Vast.ai, protegidos solo por Basic Auth: **las credenciales viajan en claro**. Es un riesgo **aceptado y documentado** para sesiones cortas con contraseña autogenerada (24 caracteres hex, única por instancia). Para uso sensible, accede por **túnel SSH** (`vastai ssh <id>` + forward local del 8188, p. ej. `ssh -L 8188:127.0.0.1:8188`) y abre `http://127.0.0.1:8188` . `ENABLE_WEB_TERMINAL=1` **(shell root) actívalo solo a través de un túnel SSH.**
- **No expongas más puertos:** solo el 8188 mapeado está previsto; no publiques otros.
- **Secretos fuera de los logs:** `COMFY_PASS` nunca se imprime si la defines tú, y `comfyctl` redacta secretos automáticamente en `/report.md` y `/status/api` . Aun así, no pegues informes en sitios públicos sin echarles un vistazo.
- **Destruye siempre:** una instancia olvidada es un gasto y una puerta abierta.

F. Checklist final imprimible

- ☑ 1. Repo en GitHub creado y `git push -u origin main` hecho (**ya hecho en este repo**)

- ☑ 2. Workflow “Build, Validate and Publish” en verde en Actions (**ya hecho**; job de build ≈ 10.5 min con `buildcache`)
- ☐ 3. Paquete GHCR `comfyuipod` puesto como **Public**
- ☐ 4. Cuenta Vast.ai creada con $5–10 de saldo
- ☐ 5. `vastai` instalado (pipx) con API key y helper `comfy` en `~/.local/bin`
- ☐ 6. `export COMFY_IMAGE=ghcr.io/<tu-usuario>/comfyuipod:latest` (y añadido a `~/.zshrc`)
- ☐ 7. `comfy rent general` → `comfy open` → pantalla de espera, `/home`, `/status` y ComfyUI OK → `comfy report` (checklist completa en `docs/auditorias/AUDITORIA_AGENTE_V5.md` §3; V1/V6/V7/V12 ya los cierra CI, GHCR Public es el paso 3)
- ☐ 8. Verificaciones: descargas reales, FATAL simulado con GPU pequeña (creada a mano: el helper filtra `gpu_ram ⩾ 24`; código 20), `comfy watch`, secretos redactados en `/report.md`, `COMFYUI_EXTRA_ARGS="--fast --cpu"` visible en `env_masked` de `/status/api` (V27/S-07 verificado)
- ☐ 9. `comfy fetch-outputs <id> <dest>` ejecutado (renders rescatados) y `comfy destroy <id>` ejecutado — instancia y cargos detenidos

---

Revisada contra la documentación y el código de `main` el 2026-11-04. Las auditorías históricas pueden describir revisiones anteriores; antes de desplegar, confirma que la CI del commit actual esté verde.